

# CS 440/540 Spring 2026

## Assignment #1: Using Lex

### Due date on Canvas

For this first assignment, you are going to use Lex (flex, jflex) to write a pretty printer for a made-up language called Rusty (which is very loosely inspired by Rust). **This is to be your own individual effort.**

The first thing to note is that you are not trying to determine if the input program is correct – you can assume it will be. In other words, you aren't writing a parser. Instead, you are going to look for particular strings and use these strings to pretty print the input program.

As an example of what pretty-printing is, suppose we have the following Rusty input with two functions test() and main():

```
fn test( ) { let mut a = 2
    + 2 ;
    let      a = 2 +
                a; println      (a); }

fn main (      ) { let mut a = 2 + 2 ; let a = 2 + a; }
```

A pretty-printed version would look something like:

```
fn test ( ) {
    let mut a = 2 + 2 ;
    let a = 2 + a ;
    println ( a ) ;
}

fn main ( ) {
    let mut a = 2 + 2 ;
    let a = 2 + a ;
}
```

The given functions now look uniform and there are uniform line breaks and spacing. Single statements are on their own lines and there is appropriate indentation. I give a larger example at the end of this document and you can see it meets the same criteria (and also shows you if and while statements and the needed nesting). Your solution doesn't have to be identical to mine (and can be nicer!) but it needs to have uniform spacing and indentation.

The two given examples in this handout tell you pretty much all you need to know about Rusty. For the purposes of this assignment, the lexical elements of importance:

- Special characters: { } ( ) , : ; = + - \* > <
- Special character strings: -> >= <= <>
- Keywords: fn let if else while println i32 mut
- Identifiers: one or more characters (a-zA-Z0-9\_) starting with an alphabetic character
- Integers are as you would expect
- Strings are strings of (non “) characters between two “ “ (“ are the character above , on a standard keyboard)
- Blanks, tabs, and statement ends (\n) are legal but characters not listed above should generate an error message.

To make your life easier, the body of `if` and `while` statements will be enclosed in { } – take my word for it, this makes it easier.

Suggestions on how to develop a solution:

- Throw away all white space in the input (just like class example) and generate all your own spacing and line breaks. This is WAY easier than keeping track of what is going on with the spacing.
- Develop your solution gradually and try out each change you write code for BEFORE writing code for another issue. For example, first write a version that throws out all the white space. Make sure that works and then write a version that does the uniform spacing between tokens.
- Focus on special tokens (like ; } {) and keywords (let, while, fn, ...) that give you insight into where to break lines. Don't worry about indenting until you can transform any Rusty input into a non-indented version with correct line breaks..

**Note** – Be sure you are using Lex effectively. If you are writing a lot of C/Java code to manipulate the matched input, you are not making Lex do the work.

## Submitting

Submit a single file named **program1.1** (if writing C code) or **program1.jflex** (if writing Java code) on Canvas/Gradescope. The output of your program should go to the screen.

- If you are submitting C, your program must compile using flex and gcc (using no library except -ll). We will run your program: `a.out < input`
- If you are submitting Java, your program must compile using the jar file given on blackboard. `java -jar ~white/JFlex.jar program1.jflex`  
`javac program1.java`  
We will run your code similarly to C: `java program1 < input`
- If there is anything else a grader would need to generate or run your code, put that information in the comment part of the submission.

```

fn test (a: i32, b:str ) -> i32 {    if (a==2) {
println(b); let c = 5; c;
} else { println(b);
    let c = 6; c;
}
    a;}

fn bigif(a: i32, b: i32, c:i32) -> i32 {
    if (a > b) {
    if (a > c) {a;}
    else {c;}
    } else if (b > c) {b;} else {c;}
}

fn whilefn(a: i32, x: str) {
    let mut b = a;
    while (b > 2) {
        println(x);
        let b = b - 1;
    }
}

fn main ( ) {
    let mut a = 2 + 2 ; let a = test(5,"test") +
a; println(test(2,"help")); whilefn(a,"again");
}

```

```

fn test ( a : i32 , b : str ) -> i32 {
    if ( a == 2 ) {
        println ( b ) ;
        let c = 5 ;
        c ;
    } else {
        println ( b ) ;
        let c = 6 ;
        c ;
    }
    a ;
}

fn bigif ( a : i32 , b : i32 , c : i32 ) -> i32 {
    if ( a > b ) {
        if ( a > c ) {
            a ;
        } else {
            c ;
        }
    } else
    if ( b > c ) {
        b ;
    } else {
        c ;
    }
}

fn whilefn ( a : i32 , x : str ) {
    let mut b = a ;
    while ( b > 2 ) {
        println ( x ) ;
        let b = b - 1 ;
    }
}

fn main ( ) {
    let mut a = 2 + 2 ;
    let a = test ( 5 , "test" ) + a ;
    println ( test ( 2 , "help" ) ) ;
    whilefn ( a , "again" ) ;
}

```